1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L29 – White Box Testing**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Testing Recap**

- TWO ways
  - Review-based – already covered earlier
  - Execution-based

5 **Execution-Based Code Testing - Recap**

- Functional testing - test cases designed using only functional specification of software, i.e. without any knowledge of internal structure [Black-box testing]
  - Already covered

6 **Execution-Based Code Testing**

- Structural testing - test cases designed using knowledge of internal structure of software [white-box testing]
  - Focus of this lecture

7

White-Box Testing

8 **Basic Idea**

- Tests internal structure of the code
- Knowledge of internal structure required
- Harder than black-box testing!

9 **Visualizing a Program**

- To understand white-box testing strategies, program (flow) visualization helps
- Can do so with Control Flow Graph (CFG)

10 **Control Flow Graph (CFG)**

11 **What?**

- Graphical representation of sequence of instruction execution
  - How the control flows through the program

12 **CFG – How to Construct?**

- Assign numbers (in sequence) to all statements of a program
- Create a 'node' in the CFG for each numbered statement

- Add an 'edge' from one node to another if execution of the statement representing first node results in transfer of control to other node

13 **CFG (Sequence)**

- Code segment
  1. A=5;
  2. B=A+10;

14 **CFG (Selection/Condition)**

- Code segment
  1. If (A>10)
     2. C = 1;
  3. Else C = 0;
  4. C=C*10;

15 **CFG (Iteration/Loop)**

- Code segment
  1. While (A<B){
     2. B = B+1;}
  3. C=A+B;

16 **Test Case Design**

17 **Test Case Design**

- Core concern – code coverage
  - Test cases should 'cover' as much code as possible
  - Coverage – extent of code accessed during execution

18 **Test Case Design**

- Many approaches
  - Statement coverage
  - Branch coverage
  - Condition coverage
  - Path coverage
  - Control flow testing
  - Data flow testing ….

19 **Test Case Design**

- Many approaches
  - Statement coverage
  - Branch coverage
  - Condition coverage
  - Path coverage
  - Control flow testing
  - Data flow testing ….

20 **Test Case Design (Statement Coverage)**

- Aims to design test cases such that every statement is executed at least once
  - Idea - unless a statement is executed, hard to determine existence of error in that

statement

- Note - executing a statement once and observing 'correct' behavior for 'some' input does not guarantee 'correct' behavior for all input values

21 **Test Case Design (Statement Coverage)**

```
int doSomething (int x, int y){
1. while (x != y){
2.     if (x > y)
3.         x = x - y;
4.     else y = y - x;
5. }
6. return x;
```

22 **Test Case Design (Statement Coverage)**

```
int doSomething (int x, int y){
1. while (x != y){
2.     if (x > y)
3.         x = x - y;
4.     else y = y - x;
5. }
6. return x;
```

23 **Test Case Design (Branch Coverage)**

- Test cases designed to cover each branch condition (both true and false values) - also known as 'edge testing'
- Guarantees statement coverage – "stronger" strategy compared to statement coverage

24 **Test Case Design (Branch Coverage)**

```
int doSomething (int x, int y){
1. while (x != y){
2.     if (x > y)
3.         x = x - y;
4.     else y = y - x;
5. }
6. return x;
```

25 **Test Case Design (Condition Coverage)**

- Test cases designed to make each component of a composite conditional expression assume both true and false values
  - E.g., ((c1 AND c2) OR c3) – test cases should make c1, c2 and c3 each assume true and false
  - n components □ 2^n test cases for each composite condition

26 **Test Case Design (Condition Coverage)**

- Branch testing – simplest condition coverage strategy (true/false values considered for whole condition rather than individual components)
- Guarantees branch and statement coverage – "stronger" strategy compared to both (may be impractical if conditions are complex)

27 **Test Case Design (Path Coverage)**

- Test cases should ensure all linearly independent paths in the code executed at least once

28 **Test Case Design (Path Coverage)**

- Path - a node and edge sequence from starting node to a terminal node of CFG of a program
  - Note – CFG can have more than one terminal node

29 **Test Case Design (Path Coverage)**

- Linearly independent path - any path with at least one new edge/node not included in any other linearly independent paths of the CFG
  - Sub path of another path not linearly independent

30 **Example**

```
int doSomething (int x, int y){
1. while (x != y){
2.     if (x > y)
3.         x = x - y;
4.     else y = y - x;
5. }
6. return x;
```

31 **Cyclomatic Complexity**

- A measure of upper bound of number of linearly independent paths for a CFG

32 **CC Computation**

- One way to compute: E-N+2
  - E = no of edges
  - N = no of nodes
- Another way: D+1 (D = no of decision statements)

33 **CC Computation - Example**

- 1st method
  - E=7, N=6
  - Complexity = E – N + 2 = 3
- 2nd method
  - D = 2 (nodes 1, 2)
  - Complexity = D+1 = 3

34 **Reference**

- Rajib Mall (2018). Fundamentals of Software Engineering, 5th ed, PHI Learning Pvt Ltd. Chapter 10
- Roger S Pressman (2020). Software Engineering: A Practitioner's Approach, 9th ed, McGraw-Hill Education, New York, Chapter 19-21